RecoverAI: Predictive Deleted File Recovery

CARP: Confidence-Adaptive Recovery Prioritization

Major Project External Viva Comprehensive Preparation Guide

Candidate: Abdul Rahamtulla Shaik
Role: Adversarial Verification
Email: shaikabdulrahamtulla@gmail.com
Affiliation: VIT-AP University (SCOPE)

Chapter 1: Role Overview & Executive Briefing

Role Definition: As the **Adversarial Verification Engineer** and ML Lead, my core responsibility is proving that RecoverAI's predictive recoverability engine remains robust, reliable, and mathematically bounded when subjected to adversarial storage conditions—including aggressive slack-space overwriting, SSD TRIM zeroing, high cluster fragmentation, and severe class imbalance (82.14% unrecoverable data).

Spoken Pitch for External Examiners (30-Second Version):

"Good morning, respected examiners. My name is Abdul Rahamtulla Shaik, and my individual responsibility is Adversarial Verification and Machine Learning Architecture. In physical digital forensics, evidence recovery fails primarily because of adversarial data decay—such as background SSD TRIM zeroing and post-deletion random slack overwriting. I developed and validated our predictive multi-task PyTorch neural network and XGBoost ensemble, achieving 82.70% classification accuracy and 0.8371 AUC-ROC under 5-fold cross-validation. Furthermore, I engineered the bounded knapsack optimization scheduler that mathematically guarantees triage operations finish within a strict 0.50 s budget per condition while pruning unrecoverable sectors."

The 1-Minute Technical Summary:

"Digital forensic examiners operate under severe time pressure on multi-terabyte drives. If an examiner blindly runs raw carving on sectors that have been zeroed by SSD TRIM commands or overwritten by system processes, hours of computational time are wasted. My role was to build the predictive verification layer that stops blind carving before it starts. I designed RecoverabilityNet, a multi-task deep neural network with a shared 3-layer representation trunk that concurrently outputs discrete recovery states and continuous byte survival fractions. Trained on our 1,260 multi-condition benchmark using a joint loss function, our soft-voting ensemble achieves an 85.40% precision on unrecoverable files. I then formulated the evidence extraction decision as a discrete bounded knapsack problem, allocating four carving tiers under a 0.50 s operational budget to achieve maximum evidentiary yield with an FRR of 15.18%."

Chapter 2: Technical Deep Dive & Mathematical Formulations

1. Multi-Task Deep Neural Architecture (RecoverabilityNet):

Rather than training separate models for classification and regression, I architected a unified multi-task network in PyTorch (pytorch_predictor.py):

- **Input Layer:** Ingests a 7-dimensional normalized feature vector: filesystem_code, discard (TRIM), occupancy_code, filler_MB, type_code, size_bytes, and elapsed_s.
- **Shared Representation Trunk:** Three dense layers (128 -> 64 -> 32 units) interleaved with Batch Normalization (stabilizes gradient flow), ReLU activations, and Dropout (rates of 0.25, 0.20, and 0.15) to prevent over-specialization on dominant unrecoverable samples.
- **Classification Head:** Linear(32 -> 3) mapping to [NOT_RETRIEVED, PARTIAL, FULL] via Softmax.
- **Regression Head:** Linear(32 -> 1) with a Sigmoid activation, constraining the predicted byte survival fraction strictly within [0.0, 1.0].

2. Joint Multi-Task Loss Formulation:

The network is trained end-to-end using a convex combination of Categorical Cross-Entropy and Mean Squared Error:

L_total = alpha * L_CrossEntropy(y_cls, y_hat_cls) + beta * L_MSE(y_reg, y_hat_reg)
*Where **alpha = 0.70** and **beta = 0.30**. Categorical discrimination receives 70% of the gradient weight because correctly identifying unrecoverable sectors is the primary operational goal in digital forensics, while regression error calibrates continuous survival ratios.*

3. Soft-Voting Ensemble Blending (XGBoost + PyTorch MLP):

Tree-based models excel at sharp step-function thresholds on tabular forensic flags (e.g., if TRIM is active, recovery probability drops to 0 immediately). In contrast, neural networks excel at smooth continuous interactions between file size, occupancy, and elapsed retention time. By computing a weighted soft-probability average across out-of-fold predictions, the ensemble reaches **0.8270 accuracy** and **0.8371 AUC-ROC**, outperforming single models.

4. Discrete Knapsack Budget Allocator:

Operational carving time is constrained to budget B = 0.50 s per condition. Each candidate file i has an effort tier e_i in {SKIP, HEADER_CHECK, SCOPED_CARVE, FULL_CARVE}. The allocator solves:
max sum(v(e_i, y_hat_reg,i)) subject to sum(c(e_i)) <= B
Where cost c(e_i) = size_MB * rate(e_i) and yield v(e_i) = y_hat * size_bytes * yield_factor(e_i). Candidates are ranked by marginal ratio r = v / c in O(N log N) greedy execution.

Model Architecture	Accuracy	Weighted F1	Macro AUC	MAE	R^2
XGBoost Baseline	0.8159	0.7904	0.8193	0.1713	0.1779
PyTorch MLP (Main)	0.8206	0.7750	0.8360	0.1739	0.2786
Ensemble (XGB + MLP)	**0.8270**	**0.7915**	**0.8371**	**0.1692**	**0.2781**

Chapter 3: Codebase Ownership & Key Functions

• pytorch_predictor.py: Implements RecoverabilityNet, 5-fold stratified training loop, EarlyStopping on validation loss, and model checkpointing.
• run_pipeline_v23.py: Master pipeline orchestrating feature engineering, ensemble blending, out-of-fold metric evaluation, and knapsack allocation execution.
• recovery_engine/orchestrator.py: Implements rank_allocations() and allocate_within_budget() enforcing the 0.50 s triage envelope.
• predictor/train.py: Trains the XGBoost baseline and exports feature importances showing post-deletion filler volume (>35% split gain) and occupancy (>20% split gain) as dominant drivers.

Chapter 4: Cross-Module Integration (How My Work Connects)

1. **Integration with Asmin (Threat Model & Recovery Engine):** Asmin generates the 24 deletion conditions and provides the true SHA-256 ground truth labels. I use these labels as targets for supervised training, and I pass my predicted fractions into Asmin's strategy cost functions.
2. **Integration with Chandrika (Semantic Search):** Chandrika's Fused Ranking Function uses my model's survival probability (P_FULL + P_PARTIAL) as a 25% weight in her search equation, ensuring corrupted files don't rank at the top.
3. **Integration with Shashwath (Database & QA):** Shashwath logs my out-of-fold prediction probabilities into the predictions table and tests my knapsack solver in tests/test_orchestrator.py.

Chapter 5: Top 25 Viva Questions & Defense Answers for Abdul

Q1: What is the primary problem you solved in this project?

I solved the blind computational inefficiency of digital forensic carving by designing an ML verification layer that predicts file recoverability before disk I/O, achieving 82.70% accuracy.

Q2: Why use a multi-task network instead of two separate models?

A multi-task network shares a single representation trunk. Learning categorical classification and continuous regression simultaneously provides regularizing cross-gradients, preventing overfitting and cutting inference latency in half.

Q3: Why is alpha=0.70 and beta=0.30 in your joint loss?

In digital forensics, discrete classification is paramount: identifying that an unrecoverable file should be skipped eliminates 100% of wasted carving. Regression acts as an auxiliary calibrator.

Q4: Why soft voting over hard voting?

Hard voting only counts discrete votes (1 vs 0), discarding prediction confidence. Soft voting averages predicted probability distributions, allowing a high-confidence prediction from PyTorch to override an uncertain prediction from XGBoost.

Q5: What was your knapsack operational budget?

B = 0.50 seconds per condition. Under this budget, the knapsack allocator assigned 843 skips, 53 scoped carves, and 364 full carves.

Q6: What is False Retirement Rate (FRR) and what was your score?

FRR is the fraction of viable files mistakenly skipped by the knapsack allocator. Our FRR was 15.18%, caused by conservative model thresholding due to the 82.14% class imbalance.

Q7: How did you prevent data leakage during 5-fold cross-validation?

StandardScaler was fit strictly on training folds and applied to validation folds. Cross-validation splits were stratified to maintain identical class ratios across all folds.

Q8: Why does XGBoost outperform PyTorch on tabular data, but the ensemble outperforms both?

XGBoost excels at discrete rule boundaries (e.g., TRIM=1 means 0% recovery). PyTorch excels at continuous smooth relationships (file size vs. decay). Blending them captures both dynamics.

Q9: What are the two most important features according to XGBoost split gain?

Post-deletion filler volume (filler_MB) and pre-deletion occupancy (occupancy_code), which together account for more than 55% of total split gain.

Q10: What is your model's precision on unrecoverable files?

0.8540 (85.40%). This high precision ensures that when RecoverAI predicts a sector is dead, it is right over 85% of the time, directly protecting investigative time.

Q11: Why is your recall on FULL files low (0.0744)?

Because only 9.61% of samples are FULL while 82.14% are NOT_RETRIEVED. Standard cross-entropy heavily favors the majority class. We will address this using Focal Loss in future work.

Q12: How does your knapsack solver guarantee it never exceeds budget B?

The greedy solver tracks cumulative incremental cost spent += c(cand) - c(current). It rejects any tier upgrade that causes spent to exceed B.

Q13: What happens if an investigator sets budget B = 0.0s?

The allocator gracefully assigns EffortLevel.SKIP to all candidates, incurring exactly 0.0s computational cost without crashing.

Q14: What activation function did you use in the regression head and why?

Sigmoid. Recovery fractions are physically bounded between 0.0 (0% bytes recovered) and 1.0 (100% bytes recovered). Sigmoid mathematically guarantees valid bounds.

Q15: Why did you use Batch Normalization in the shared trunk?

Batch Normalization reduces internal covariate shift and smooths the optimization landscape, allowing faster convergence across divergent feature scales.

Q16: How did you select the dropout rates (0.25, 0.20, 0.15)?

We used a tapering dropout architecture: higher dropout in the early wide layers (128 units) to prevent co-adaptation, and lower dropout in the bottleneck layer (32 units) to preserve latent features.

Q17: What was the stopping epoch during 5-fold cross validation?

Early stopping monitored validation loss with a patience of 15 epochs. Models stopped between epoch 32 and epoch 63, preventing validation overfitting.

Q18: What was the mean validation accuracy across all 5 folds?

0.8206 +/- 0.0095, demonstrating exceptional stability across different data splits.

Q19: How did you evaluate R^2 for regression?

R^2 was evaluated out-of-fold. The ensemble achieved R^2 = 0.2781 with an MAE of 0.1692 bytes fraction error.

Q20: How does your model handle adversarial random overwrite filler?

filler_MB is an explicit input feature. The model learns the empirical threshold where filler volume exceeds cluster slack, dynamically depressing recovery confidence.

Q21: How does your work integrate with Chandrika's semantic search?

My model outputs P_FULL + P_PARTIAL, which Chandrika's Fused Ranking Function uses as a 25% weight in ranking recovered evidence.

Q22: How does your work integrate with Asmin's recovery engine?

My knapsack decisions directly drive Asmin's xCarver execution depth: files assigned SKIP are never touched, while FULL_CARVE files undergo complete sector extraction.

Q23: How does your work integrate with Shashwath's database layer?

All out-of-fold predictions and knapsack allocations are permanently persisted in Shashwath's SQLite schema for forensic auditability.

Q24: What baseline literature did you compare against?

We benchmarked against Yermolov (2025), who achieved ~80% accuracy. Our ensemble reached 82.70% accuracy and 0.8371 AUC-ROC on physical disk samples.

Q25: What is the single biggest technical improvement you would make?

Replacing cross-entropy loss with Class-Balanced Focal Loss to suppress the False Retirement Rate below 5% on highly skewed media.

Chapter 6: Rapid Revision Cheat Sheet for Abdul

- ***Ensemble Accuracy:*** *0.8270 (82.70%)* | ***Macro AUC-ROC:*** *0.8371* | ***MAE:*** *0.1692* | ***R^2:*** *0.2781*
- ***Dataset:*** *N = 1,260 samples (360 empirical disk traces + 900 forensically grounded records)*
- ***Class Distribution:*** *1,035 NOT_RETRIEVED (82.14%), 104 PARTIAL (8.25%), 121 FULL (9.61%)*
- ***Knapsack Budget:*** *B = 0.50 s* | ***Allocations:*** *843 skips, 53 scoped carves, 364 full carves* | ***FRR:*** *15.18%*
- ***Loss Formula:*** *L_total = 0.70 * CrossEntropy + 0.30 * MSE*
- ***Trunk Architecture:*** *Linear(7,128)->BN->ReLU->Drop(0.25)->Linear(128,64)->BN->ReLU->Drop(0.20)->Linear(64,32)->BN->ReLU->Drop(0.15)*
- ***Key Code:*** *pytorch_predictor.py, run_pipeline_v23.py, recovery_engine/orchestrator.py*